

Design of a ROM-Based Hexadecimal Digital Clock Using ALU in Logisim

(Semester Project)



Submitted By:

- Muhammad Hammad (BSCS-023R23P-52)
- Muhammad Umair Ullah (BSCS-023R23P-58)
- Muhammad Khurram (BSCS-023R23P-82)

BSCS 5th : Computer Architecture (Blue)

Submitted To:

Ma'am Sana Fatima

DEPARTMENT OF COMPUTER SCIENCE

UNIVERSITY OF SOUTHERN PUNJAB.MULTAN

Design of a ROM-Based Hexadecimal Digital Clock Using ALU in Logisim

Objective

- To design and simulate a hexadecimal digital clock using **ROM and ALU** in Logisim
- To understand **ROM-based next-state logic** instead of direct counting
- To implement **hexadecimal counting (0–F)** with automatic reset
- To strengthen concepts of **computer organization and architecture** through simulation

Introduction

- In this project, I designed a hexadecimal digital clock using a **ROM-based approach** instead of a simple counter.
- The system uses a **clock signal** to control time progression and an **ALU** to perform increment operations.
- A **ROM** stores predefined next-state values, ensuring controlled state transitions.
- The output is displayed in **hexadecimal format (0–F)** using a Hex Digit (7-segment) display.

Components Used

- Clock
- ROM (4-bit address, 4-bit data)
- ALU
- Register

- Counter / Address Logic
- Hex Digit Display (7-segment)
- Reset Button
- Wires

Theory Background

Read Only Memory (ROM)

- ROM is a **non-volatile memory** used to store permanent data
- Data stored in ROM does not change during operation
- ROM is commonly used for:
 - Lookup tables
 - Control units
 - Fixed control values

ALU (Arithmetic Logic Unit)

- ALU performs arithmetic and logical operations
- In this project, ALU is used to **increment the current hexadecimal value**

Hexadecimal Number System

- Uses base-16 values: **0–9 and A–F**
- After reaching **F (15)**, the counter resets to **0**

Functional Description

- The current hexadecimal value is stored in a **register**
- The register output is sent to:
 - ROM as an **address input**

- ALU for **increment operation**
- ROM provides the **next-state value** based on the current address
- ALU increments the value
- Updated value is stored back into the register on the next clock pulse
- The output is displayed on a **Hex Digit Display**

Procedure (Step-by-Step)

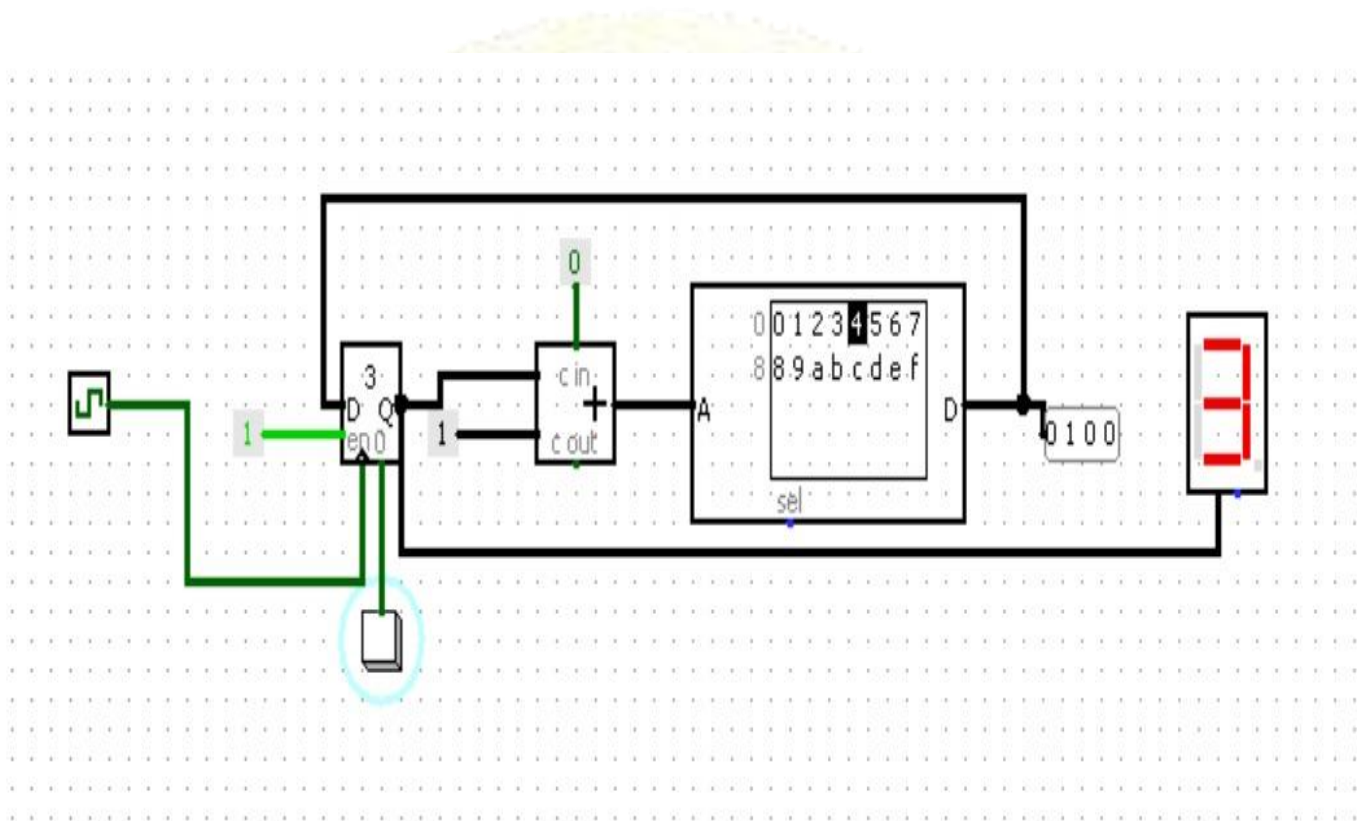
1. Place a **clock** component to generate timing pulses
2. Connect a **register** to store the current hexadecimal value
3. Connect the register output to the **ROM address input**
4. Configure ROM with predefined next-state hexadecimal values
5. Connect the register output to the **ALU input**
6. Set ALU operation to **increment mode**
7. Feed ALU output back to the register input
8. Connect a **reset button** to initialize the register to 0
9. Connect register output to **Hex Digit Display**
10. Verify that values increment from 0 to F
11. Ensure the system resets to 0 after F
12. Simulate and observe correct hexadecimal counting

Simulation / Result Description

- On each clock pulse, the hexadecimal value increments by one
- ROM correctly outputs predefined next-state values
- ALU performs accurate increment operation

- Hex Digit Display shows values from **0** to **F**
- After reaching **F**, the system resets to **0**

Circuit:



Observations

- ROM output depends only on the address input
- ROM data remains constant throughout operation
- ALU successfully performs arithmetic increment
- Clock controls synchronized data flow
- System behaves as expected for all hexadecimal values

Applications

- Digital clocks
 - Control units
 - ROM-based state machines
 - Embedded system timing circuits
-

Conclusion

- I successfully designed and simulated a **ROM-based hexadecimal digital clock** using ALU in Logisim.
- The project improved my understanding of **ROM-based control logic and arithmetic operations**.
- Using ROM for next-state logic provides better control and scalability.
- This project effectively demonstrates core concepts of **Computer Organization and Architecture**.